

radcoolpv

A YAML-driven radiative-cooling photovoltaics simulator
User manual

Version 0.1.0

Contents

1	Overview	2
2	Installation and requirements	2
2.1	Requirements	2
2.2	Install	2
2.3	Which interpreter runs the code (important)	2
3	Project structure	2
4	Running	3
5	YAML configuration reference	3
5.1	run	3
5.2	simulation	4
5.3	geometry	4
5.4	structure	4
5.5	materials	5
5.6	thermal	5
5.7	data	5
6	Inputs and outputs	5
7	Error messages and common problems	6
8	Worked examples	6
8.1	Free-form optics + PV (no live RCWA)	6
8.2	Cooling-power curve (Validation B)	6
8.3	Resume from a reduced spectrum	6
8.4	Live grcwa optics (small)	7
9	Supported calculations	7
9.1	Optical calculations	7
9.2	PV-module (thermal / electrical / radiative-cooling) calculations	8
10	Running the trusted validations	8
11	Limitations of the current version	8

1 Overview

`radcoolpv` is a pure-Python simulator for the radiative cooling and photovoltaic performance of photonic-structured silicon solar modules. A single YAML file drives a two-stage pipeline:

1. **Optics** — spectral and directional reflectance, transmittance, absorptance and silicon-layer absorptance of a photonic structure on a multilayer stack, computed by rigorous coupled-wave analysis (RCWA) with the pure-Python `grcwa` engine, read from free-form data, or resumed from a previous run.
2. **Thermal / electrical** — the steady-state energy balance (radiative, atmospheric, convective, solar, and, in the full-PV mode, electrical and luminescence terms), the equilibrium cell temperature, and the diode I-V / MPP / V_{oc} / fill-factor / efficiency.

The two stages share one wavelength grid and are coupled in memory. Everything in this manual reflects the behaviour actually implemented in the code; planned or unsupported options are called out as such.

2 Installation and requirements

2.1 Requirements

- Python ≥ 3.9 .
- The Python packages `numpy`, `scipy`, `matplotlib`, `pyyaml`, `openpyxl`, and `grcwa` (all pure Python). `pytest` and `tmm` are needed only to run the test suite.
- No compiled or native dependency — the RCWA engine (`grcwa`) is pure Python, so a plain `pip` install covers the whole toolchain.

2.2 Install

From the repository root:

```
./install.sh          # create a .venv and install everything into it
./install.sh --system # install into the current Python instead of a venv
```

or by hand:

```
pip install -r requirements.txt && pip install -e .
```

2.3 Which interpreter runs the code (important)

If a stale editable install of `radcoolpv` exists elsewhere on the machine, the `radcoolpv` console script (and any script launched *by path*) can import that stale copy instead of your working tree — symptoms include a `TypeError` about an unexpected `cooling_temperature` argument. Pin the working copy by prefixing commands with `PYTHONPATH="$PWD"` (run from the repository root), or re-run `pip install -e .` from the working copy.

3 Project structure

```

radcoolpv/
cli.py      'radcoolpv run ...' entry point
config.py   YAML -> typed config + validation + derived helpers
constants.py physical constants / unit conversions
pipeline.py orchestrator (stage coupling, outputs, plots)
materials/  registry + tabulated loader + analytic models + data/ (CSV n,k)
optics/     geometry, grcwa_backend, directional, averages, freeform
thermal/    spectra, radiative, pv, energy_balance
io/         results context + legacy/clean writers
plotting/   figures
data/       bundled solar / atmosphere / IQE / free-form example
configs/    example YAML configs
validations/ literature-reproduction checks (A and B are trusted)
tests/      pytest suite

```

4 Running

```

radcoolpv run configs/freeform.yaml          # optics + PV
radcoolpv run configs/full.yaml --print-config # show resolved settings only

```

The only subcommand is `run`; `--print-config` parses and prints the resolved configuration and exits without computing. Each run creates a timestamped folder under `results_dir/` (see Section 6).

5 YAML configuration reference

A configuration has the top-level sections `run`, `simulation`, `geometry`, `structure`, `materials`, `thermal`, `data`, and (only for `spectral_compare`) `comparison`. Every field below is optional unless marked required; omitted fields take the default shown.

5.1 run

Field	Type	Default	Meaning / values
<code>optics</code>	bool	<code>true</code>	Run the optics stage.
<code>thermal</code>	bool	<code>true</code>	Run the thermal stage.
<code>plots</code>	bool	<code>true</code>	Write PNG figures.
<code>mode</code>	enum	<code>standard</code>	<code>standard</code> <code>cooling_curve</code> <code>test</code> <code>spectral_compare</code> . (<code>test2</code> is accepted but <i>not implemented</i> — it behaves like <code>standard</code> ; do not use it.)
<code>results_dir</code>	path	<code>results</code>	Parent folder for output.
<code>outputs</code>	list	<code>[legacy, clean]</code>	Any of <code>legacy</code> , <code>clean</code> .
<code>optics_results</code>	path	<code>null</code>	When <code>optics: false</code> , resume from this raw <code>OUTPUTS4</code> folder or reduced-spectrum file.

5.2 simulation

Field	Type	Default	Meaning / values
wavelength.min	float	0.3	Grid start, μm (> 0).
wavelength.max	float	30.0	Grid end, μm .
wavelength.n	int	2000	Number of points (≥ 2); grid is <code>linspace(min,max,n)</code> .
angles	enum	hemispherical	normal (0°) or hemispherical (18 angles, $0-85^\circ$).
rcwa_modes	int	10	grcwa Fourier truncation <code>nG</code> . Patterned structures typically need $\sim 100+$.
grid_nx	int	128	Raster resolution of patterned layers (grcwa).
grid_ny	int	128	Raster resolution of patterned layers (grcwa).

5.3 geometry

Field	Type	Default	Meaning / values
source	enum	grcwa	grcwa (live RCWA) or freeform (read a data file).
shape	enum	flat	flat sphere semisphere triangle cylinder.
photonic_material	name	sio2	Logical material of the photonic pattern.
lattice.type	enum	square	square or hexagonal.
lattice.x, .y	float	20.0	Lattice pitch, μm (square uses <code>x</code> for both axes).
discretization_layers	int	1	Slices for sphere/semisphere/triangle.
sphere.radius	float	—	μm (required for sphere/semisphere).
triangle.base, .height	float	—	μm (required for triangle).
cylinder.radius, .height	float	—	μm (required for cylinder).
freeform.file	path	—	Data file (required when <code>source: freeform</code>).

5.4 structure

An ordered list of flat layers below the photonic structure, top to bottom. Each item is `{material, thickness, terminal}`: `material` is a logical name declared in `materials`; `thickness` is in μm ; exactly one layer must set `terminal: true` (the semi-infinite substrate). Exactly one layer should use material `silicon` — it is the single source of truth for the cell thickness, and its per-layer absorptance drives the photocurrent.

5.5 materials

A mapping from logical name to a registry model. Tabulated (λ, n, k) models are bundled as CSVs: PalikKitamura_SiO2, SiliconNew, Hagemann_Ag, Rubin_SodaLime, GuptaQuerry_PDMS, Jaramillo_NILresist. Analytic models: Vacuum, DrudeSi3N4. The name vacuum is always available ($\varepsilon = 1$). Add a material by dropping a `lambda_um,n,k` CSV into `radcoolpv/materials/data/`.

5.6 thermal

Field	Type	Default	Meaning / values
<code>ambient_temperature</code>	float	298	T_{amb} , K.
<code>convection_coefficient</code>	float	12	Non-radiative coefficient h_c , W/m ² K.
<code>voltage.min/max/n</code>	—	0.1/0.8/100	PV bias sweep, V / count.
<code>equilibrium</code>	enum	auto	auto (fixed-point solve) or manual (use <code>emit_temp</code> , <code>vmpp</code>).
<code>cooling_temperature</code>	sweep	—	{min,max,n} in K; <i>required</i> for <code>cooling_curve</code> .
<code>solar_irradiance</code>	float	null	<code>cooling_curve</code> only: rescale absorbed solar to this W/m ² .
<code>reference_curve_file</code>	path	null	<code>cooling_curve</code> plot overlay (digitized reference).
<code>reference_curve_column</code>	int	1	Column of the overlay file to plot.
<code>emit_temp</code>	float	319	Equilibrium T for <code>equilibrium: manual</code> .
<code>vmpp</code>	float	0.6586	MPP voltage for <code>manual</code> .
<code>pv.series_resistance</code>	float	1.1e-4	R_s , Ω m ² .
<code>pv.shunt_resistance</code>	float	0.1	R_{sh} , Ω m ² .
<code>pv.bandgap</code>	—	see code	{ <code>eg0,alpha,beta</code> } of $E_g(T) = e_{g0} - \alpha T^2 / (T + \beta)$.
<code>pv.iqe_file</code>	path	<code>data/siliconIQE.txt</code>	IQE table quantum efficiency table.

5.7 data

`solar_spectrum` (default `data/astmg173.xlsx`, AM1.5G) and `atmosphere` (default `data/cptrans_nq_100_15.dat`, Cerro-Pachón transmittance). Paths are resolved relative to the config, then to the package's bundled `data/`, so the defaults resolve by basename.

6 Inputs and outputs

Required inputs. The optical description (live geometry + materials, a free-form file, or a resume target), the solar spectrum, the atmospheric transmittance, and — for the full-PV path — the IQE table and PV parameters.

Outputs (per run, in a timestamped folder). With `legacy`: `OUTPUTS4-TE.txt` / `OUTPUTS4-TM.txt` (raw per-angle optics), `opticalProps-PVcode.txt`, `IV-PVcode.txt`, `Power-PVcode.txt`, `simulParam.log`

/ simulParamPV.log. With clean: `optics.csv`, `iv.csv`, `power.csv` (full PV) or `cooling_power.csv` (`cooling_curve`), and `run.json` (a record of key inputs and scalar results: T_{eq} , V_{mpp} , I_{sc} , MPP, V_{oc} , FF, P_{atm} , absorbed P_{sun} , efficiency, β_P). Figures are written to `figures/` when `plots: true`.

7 Error messages and common problems

- `TypeError: ... unexpected keyword argument 'cooling_temperature'` — a stale install is being imported; see Section 2.3 (use `PYTHONPATH`).
- `ConfigError: run.mode must be ... /geometry.source must be 'grcwa' or 'freeform' / 'structure' must mark exactly one layer as terminal` — configuration validation; the message names the offending field.
- `ConfigError: cooling_curve mode requires thermal.cooling_temperature with max > min and n >= 2`.
- `RuntimeError: The grcwa module is not installed — pip install grcwa`.
- `... expected 5-column HEMSIPH or 7-column PVcode reduced optics` — a resumed reduced-spectrum file has the wrong number of columns.
- A live patterned RCWA sweep is slow (pure Python): reduce `wavelength.n`, set `angles: normal`, or lower `rcwa_modes`.

8 Worked examples

8.1 Free-form optics + PV (no live RCWA)

`configs/freeform.yaml` reads a precomputed free-form spectrum and runs the full PV energy balance:

```
PYTHONPATH="$PWD" radcoolpv run configs/freeform.yaml
```

Produces `optics.csv`, `iv.csv`, `power.csv`, `run.json` and `figures`; the log prints T_{eq} , V_{mpp} and the MPP.

8.2 Cooling-power curve (Validation B)

```
PYTHONPATH="$PWD" radcoolpv run "validations/validation B/fig4d_pdms.yaml"
```

A `cooling_curve` run: writes `cooling_power.csv` and a cooling-curve figure and prints the equilibrium temperature (here $T_{eq} \approx 329.9$ K).

8.3 Resume from a reduced spectrum

Set `run.optics: false` and point `run.optics_results` at a 5-column (`lambda,R,T,emit,abs_si`) or 7-column reduced file; the thermal stage runs directly on it. This is how Validations A and B operate.

8.4 Live grcwa optics (small)

A minimal patterned run (normal incidence, few wavelengths) exercises the RCWA engine end-to-end:

```
run: {optics: true, thermal: false, plots: false, mode: standard,
      results_dir: results, outputs: [clean]}
simulation: {wavelength: {min: 4.0, max: 12.0, n: 6}, angles: normal,
            rcwa_modes: 40, grid_nx: 64, grid_ny: 64}
geometry:
  source: grcwa
  shape: cylinder
  photonic_material: sio2
  lattice: {type: hexagonal, x: 6.125, y: 6.125}
  cylinder: {radius: 1.75, height: 2.25}
structure:
  - {material: sio2, thickness: 50.0}
  - {material: silicon, thickness: 3.0}
  - {material: substrate, thickness: 0.0, terminal: true}
materials: {sio2: PalikKitamura_SiO2, silicon: SiliconNew,
            substrate: Hagemann_Ag}
data: {solar_spectrum: data/astmg173.xlsx,
       atmosphere: data/cptrans_nq_100_15.dat}
```

9 Supported calculations

This section lists only what the current code implements.

9.1 Optical calculations

- **Engines / inputs.** (i) Live RCWA via the pure-Python `grcwa` engine (`source: grcwa`); (ii) free-form data loader (`source: freeform`, an 8-column normal-incidence file); (iii) resume from a raw `OUTPUTS4` folder or a 5-/7-column reduced spectrum.
- **Structures.** flat, sphere, semisphere, triangle, cylinder (spheres/triangles staircase-discretised), on an ordered flat `structure` with exactly one terminal substrate; square or hexagonal lattice.
- **Materials.** Tabulated (n, k) CSVs and analytic models (`Vacuum`, `DrudeSi3N4`); dispersive, complex (absorbing) permittivity.
- **Wavelength / angle / polarisation.** `linspace(min, max, n)`; normal (0°) or hemispherical (18 angles, $0-85^\circ$); TE at normal incidence, TE + TM for hemispherical (handled internally).
- **Outputs.** Reflectance R , transmittance T , total absorptance $A = 1 - R - T$ (used as emittance), and silicon-layer absorptance, each with hemispherical and normal-incidence copies; the atmospheric emissivity and trapezoidal band averages.
- **Backend.** All live RCWA is `grcwa`. In `grcwa`, patterned layers are rasterised to an $N_x \times N_y$ grid and `rcwa_modes` sets the Fourier truncation; silicon absorptance is the net Poynting-flux difference across the silicon layer.
- **Assumptions / limits.** One terminal (semi-infinite) substrate; $\cos\theta\sin\theta$ hemispherical quadrature; staircase rasterisation of patterns; pure-Python performance (see limitations).

9.2 PV-module (thermal / electrical / radiative-cooling) calculations

- **Modes.** `standard` (full PV energy balance + I-V), `cooling_curve` (PV-free radiative balance over a temperature sweep), `test` (Perrakis 2020 reference with a fixed absorbed-solar value), `spectral_compare` (overlay plot only).
- **Energy-balance terms.** $P_{rad} = \pi \int \varepsilon B_{\lambda}(T) d\lambda$; $P_{atm} = \pi \int \varepsilon \varepsilon_{atm} B_{\lambda}(T_{amb}) d\lambda$; $P_{conv} = h_c(T - T_{amb})$; P_{sun} from $\int \varepsilon I_{AM1.5G} d\lambda$ (optionally rescaled by `solar_irradiance`); and, in `standard`, the electrical P_{mpp} and luminescence $P_{nonthermal}$. Equilibrium is the first zero-crossing of the cooling curve.
- **Electrical model.** Temperature-dependent bandgap; J_{sc} from IQE $\times$ silicon absorptance $\times$ solar photon flux up to λ_g ; saturation current from the blackbody photon flux; Auger recombination; an R_s/R_{sh} diode solved per bias with `fsolve`; MPP, V_{oc} , fill factor, efficiency (MPP / total AM1.5G) and the temperature coefficient β_P .
- **Required parameters.** T_{amb} , h_c ; optics (emittance + silicon absorptance); AM1.5G spectrum; atmospheric transmittance; and, for `standard`, the IQE table, R_s , R_{sh} and the bandgap.
- **Outputs.** T_{eq} ; and for `standard`: V_{mpp} , I_{sc} , MPP (ambient and equilibrium), V_{oc} , FF, efficiency, β_P , P_{atm} , absorbed P_{sun} , equilibrium radiated power; for `cooling_curve`: the cooling-power curve and T_{eq} .
- **Assumptions / approximations.** Convection linear in ΔT ; conduction to the mount neglected; a single silicon layer; a single scalar bandgap wavelength per run (MATLAB-parity reduction); efficiency normalised to total (not absorbed) AM1.5G; the hemispherical `emit_atm` output column is an unweighted angular integral (display only).

10 Running the trusted validations

From the repository root (see `validations/README.md`):

```
# Validation A -- Silva-Delker & Jaramillo-Fernandez 2022, Table 1 (full PV)
PYTHONPATH="$PWD" python "validations/validation A/run_table1_validation.py"

# Validation B -- Le et al. 2026, cooling-power curves
for f in fig4d_pdms fig4d_sds fig4d_ads fig5d_cooling_family; do
    PYTHONPATH="$PWD" radcoolpv run "validations/validation B/$f.yaml"
done
```

Both resume from precomputed optical spectra, so they do not invoke the RCWA engine. Validation A prints a Published / Computed / Rel-err table (most rows within $\sim 2\%$); Validation B writes cooling curves and prints equilibrium temperatures.

11 Limitations of the current version

- Live patterned RCWA is pure Python and therefore much slower than the former C++ S4 backend; patterns are rasterised (staircase), and `nG` / grid resolution must be chosen for convergence.
- `run.mode:` `test2` is accepted by the parser but not implemented (it behaves like `standard`).
- Efficiency is normalised to the total AM1.5G power, not the absorbed power.

- The hemispherical `emit_atm` output column is an unweighted angular integral (it maxes near $\pi/2$); it is a display quantity, not an emissivity.
- Reproducibility depends on importing the working copy of the package (Section 2.3).